

```
const classifier = ml5.imageClassifier('MobileNet', function()
{
  console.log('Model Loaded!');
});
// Make a prediction with the selected image
// This will return an array with a default of 10 options
with their probabilities
classifier.predict(image, function(err, results) {
  result.innerText = results[0].className;
  probability.innerText = results[0].probability.toFixed(4);
});

</script>
</body>
</html>
(Source: https://ml5js.org/docs/getting-started.html)
```

Just save this file and open it in a browser. Your working demo is ready. There are no dependencies to configure. It can be inferred from the above code that ml5.js has direct functions available to perform the classification task. We just need to specify the model and the results are ready.

The official documentation has listed various code examples such as a style transfer, a Pong game in the browser, and a text editor with ML based suggestions (<https://ml5js.org/experiments>).

Brain.js

This is a useful JavaScript ML library. It is used to build neural networks in the browser. The 'node' based configuration of Brain.js can be done as follows:

```
npm install brain.js
```

...or by using the following command:

```
yarn add brain.js
```

The types of neural networks supported by Brain.js are given below:

- Feed forward neural network: *brain.NeuralNetwork*
- FF neural networks with back-propagation: *brain.NeuralNetworkGPU*
- Time step recurrent neural network (NN) or RNN: *brain.recurrent.RNNTimeStep*
- Time step long/short term memory (LSTM) NN: *brain.recurrent.LSTMTimeStep*
- Time step gated recurrent unit (GRU): *brain.recurrent.GRUTimeStep*
- RNN: *brain.recurrent.RNN*
- LSTM neural network: *brain.recurrent.LSTM*
- GRU: *brain.recurrent.GRU*

Further details on Brain.js can be obtained from <https://github.com/BrainJS/brain.js>.

ConvNetJS

This is an implementation of neural networks in JavaScript. ConvNetJS supports the following:

- Common NN modules (full connected layers and non-linearities)
- Classification and regression
- Can train convolutional neural networks for processing images
- A reinforcement learning module based on Deep Q learning

ConvNetJS has no external dependencies and hence is very simple to handle. The official documentation lists many working demos. A CIFAR-10 demo is provided at <https://cs.stanford.edu/people/karpathy/convnetjs/demo/cifar10.html>.

Importing ConvNetJS is done with the following simple script tag:

```
<!-- import convnetjs library -->
<script src="convnet-min.js"></script>
```

Multi-layer networks can be built with the following code sequence:

```
var layer_defs = [];
// input layer of size 1x1x2 (all volumes are 3D)
layer_defs.push({type:'input', out_sx:1, out_sy:1, out_
depth:2});
// some fully connected layers
layer_defs.push({type:'fc', num_neurons:20,
activation:'relu'});
layer_defs.push({type:'fc', num_neurons:20,
activation:'relu'});
// a softmax classifier predicting probabilities for two
classes: 0,1
layer_defs.push({type:'softmax', num_classes:2});
// create a net out of it
var net = new convnetjs.Net();
net.makeLayers(layer_defs);
(Source: https://cs.stanford.edu/people/karpathy/convnetjs/started.html)
```

The libraries mentioned in this article enable the developer to incorporate ML into Web applications without many dependencies. Browser based ML can harness the untapped hardware potential on the client side instead of loading the server heavily with a huge number of requests. As these libraries advance further, future Web applications will be able to provide various smart features based on ML algorithms that will benefit users greatly. **END** 

 By: Dr K.S. Kuppusamy

The author is assistant professor of computer science at the Pondicherry Central University. He has over 13 years of teaching and research experience in academia and industry. He can be reached at kskuppu@gmail.com.